MarIA - Arquitetura do HUB de integração

Versão 1.1 | 6º período, Fase 02 | 11/10/2026

20. Arquitetura geral do sistema

O MarIA é um serviço de integração para continuidade do atendimento entre clínicas. Os estabelecimentos continuam utilizando seus próprios softwares, que enviam registros ao HUB e consultam o histórico permitido por sua API. A clínica é o tenant; o paciente possui identidade no HUB; o médico é um usuário vinculado a uma ou mais clínicas. O acesso entre tenants é uma exceção explícita governada por autorizações, não uma abertura global dos dados.

O frontend existente é reaproveitável como sistema de clínica de demonstração. Uma tela adicional em /demo/ representa dois contextos clínicos e o portal do paciente. Não há integração comprovada com fornecedores externos nesta etapa. Cada fornecedor exigirá avaliação de API, contrato, adaptador e qualidade dos dados.

20.1 Componentes

• Sistemas das clínicas e adaptadores: traduzem dados locais para o contrato do HUB. O adaptador inicial é a tela demonstrativa que envia JSON. Integrações comerciais são futuras.

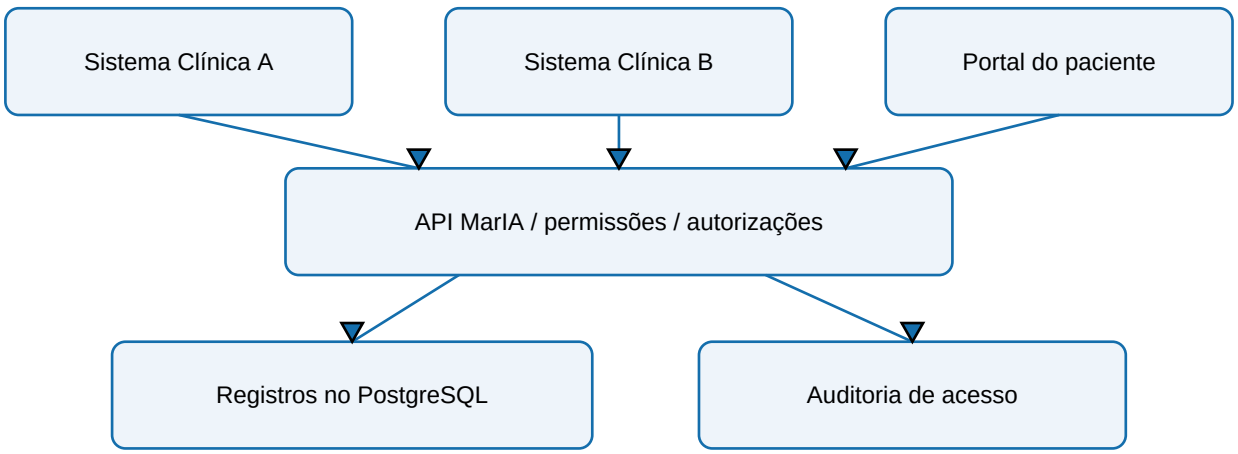
• API MarIA: Django REST Framework; autenticação do usuário, identificação do sistema conectado, validação de vínculo e papel, ingestão de registros e consulta autorizada.

• Portal do paciente: mostra finalidade, destinatário, versão do termo e prazo; permite aceitar, recusar e revogar. Nesta etapa opera com contas fictícias provisionadas.

• PostgreSQL: armazenamento central de cópias dos registros recebidos e de seus identificadores de origem, vínculos, autorizações e auditoria.

• Auditoria: registra decisões, consultas permitidas e bloqueios de vínculo, sem copiar o conteúdo clínico para o log.

20.2 Visão lógica



20.3 Tenancy, identidade e persistência

Adota-se um banco com tabelas compartilhadas. Clínica é a fronteira organizacional. Não são utilizados schemas por médico nem django-tenant-schemas. O acesso é controlado no servidor por vínculo ativo, papel assistencial, clínica de origem e autorização vigente. Filtros de listagem são tão importantes quanto verificações de detalhe.

Paciente representa uma identidade global do HUB; PatientLink relaciona essa identidade à clínica e ao identificador local. O protótipo provisiona esses vínculos por comando administrativo, sem pesquisa global por CPF. Cadastro público, confirmação de identidade real, conciliação de duplicidades e representantes legais não estão implementados.

Modelo inicial de entidades

• Clinic: identificador UUID e nome do tenant.

• Membership: usuário, clínica, papel e estado ativo. Um usuário pode ter mais de um vínculo.

• Patient: UUID, conta própria, nome e nascimento fictícios.

• PatientLink: paciente, clínica e identificador externo; unicidade por clínica/paciente e por clínica/identificador externo.

• Integration: clínica, nome, estado ativo e hash da credencial do sistema.

• Record: integração de origem, paciente, usuário autor, identificador externo, data do atendimento, resumo e data de recebimento.

• Grant: paciente, clínica destinatária, solicitante, finalidade, escopo, prazo, estado, texto e versão do termo, datas da decisão e revogação.

• Audit: usuário, clínica, paciente, ação, resultado, identificadores dos registros e data.

O banco central recebe uma cópia selecionada, não substitui automaticamente o prontuário de origem. A leitura preserva autoria e clínica de origem; o destinatário não altera registros externos. No contrato inicial, o par integração/identificador externo evita duplicidades: reenvio idêntico retorna o existente; conteúdo divergente retorna 409. Correções versionadas são uma pendência, não sobrescritas silenciosas.

Regra de autorização adotada no protótipo

O paciente autoriza a equipe assistencial habilitada de uma clínica para uma finalidade e um prazo. Não é necessária aprovação manual da clínica ou do profissional de origem a cada atendimento. Os acordos de participação, responsabilidades e fundamentos para disponibilização de registros devem ser estabelecidos antes de uso real.

O escopo inicial é prior_records: registros ocorridos e recebidos até o momento do aceite. Novos registros após o aceite não estão incluídos; exigem nova solicitação. A consulta exige validade vigente, estado aprovado e ausência de revogação. O vencimento é calculado no momento da consulta, sem depender de tarefa agendada. Recepção não consulta conteúdo clínico.

21. Tecnologias e ferramentas

• Python 3.12: ambiente local de desenvolvimento do backend.

• Django 5.2 e Django REST Framework 3.16: modelos, migrations, APIs e validações.

• SimpleJWT: tokens de acesso de 15 minutos e renovação de 8 horas para o protótipo. Vínculos e autorizações são consultados no banco; claims do token não substituem essas verificações.

• PostgreSQL 17 e psycopg: destino de persistência; serviço configurado no Docker Compose, com volume persistente e porta restrita ao computador local.

• SQLite: opção explícita MARIA_TEST_SQLITE=1 para testes locais enquanto PostgreSQL não estiver disponível. Não equivale a validar comportamento no PostgreSQL.

• HTML, CSS e JavaScript/Fetch: cliente de demonstração e portal do paciente. Tokens ficam em memória nesta tela, sem localStorage. A página e a API usam a mesma origem, dispensando CORS nesta demonstração.

• Git e arquivos Markdown: versionamento e fonte editável da documentação. ReportLab é usado apenas para produzir este PDF.

Segurança e limites

Credenciais de integração são armazenadas em hash. A ingestão exige chave do sistema e JWT de profissional com vínculo assistencial na clínica dessa integração; autoria é obtida do usuário autenticado, não de um ID livre no corpo. UUID não é uma permissão.

O ambiente é acadêmico, com dados fictícios. HTTPS, controle de segredos em produção, políticas de retenção, backups, monitoração, limitação de tentativas, bloqueio/rotação de sessões, identidade verificada e governança institucional permanecem requisitos para uso real. Não se afirma certificação LGPD, ISO 42001, assinatura ICP-Brasil ou criptografia AES-256 implementada.

Revogar encerra novos acessos sustentados por aquela autorização, mas não elimina automaticamente cópias legitimamente incorporadas ao atendimento destinatário. Se houver outra autorização vigente, seu escopo ainda pode sustentar acesso; o portal apresenta cada autorização separadamente.

IA, emissão de receitas, pagamentos, agenda, fila de espera, Celery/Redis e aplicativo móvel ficam fora do núcleo desta fase. Poderão ser extensões, mediante justificativa e integração efetiva.

22. Implementação por camadas

22.1 Frontend

A tela /demo/ possui login real pela API, seleção de clínica vinculada, listagem local de pacientes, envio de atendimento, solicitação de acesso, consulta ao histórico e portal do paciente para decisão. É um cliente de demonstração, não uma integração comercial.

As telas HTML originais permanecem como protótipo de sistema clínico e ainda usam dados locais. Não se declara que cadastro, agenda, dashboard ou triagem originais estejam integrados. A migração dessas telas será incremental a partir do contrato de API implementado.

22.2 Backend

Os módulos iniciais ficam em backend/config e backend/hub. Incluem modelos relacionais, migration inicial, serializers, endpoints, comando seed_demo e testes. A subdivisão por domínios pode ocorrer quando o tamanho e a complexidade a justificarem.

• Autenticação: login e renovação JWT; endpoint me informa identidade e vínculos.

• Ingestão: valida sistema, profissional, paciente local, data e idempotência.

• Autorizações: solicitação por profissional da clínica vinculada, decisão somente pelo próprio paciente, termo versionado e revogação.

• Histórico: registros da própria clínica e registros externos abrangidos por autorização; procedência mantida e auditoria da consulta.

• Provisionamento: seed_demo cria clínicas A/B, médicos, paciente e recepção com senha definida no ambiente; reexecutar rotaciona credenciais de demonstração.

Não há cadastro público de pacientes ou clínicas, inclusão automática de vínculos, anexos, correções de registros ou adaptadores comerciais nesta versão. O backend não oferece edição de registros clínicos já recebidos.

22.3 Integração frontend e backend

1. Clínica A: o usuário faz login, obtém vínculos e envia JSON ao endpoint de integração com JWT e chave do sistema. O servidor determina origem e autoria e persiste o registro.

2. Clínica B: um profissional vinculado solicita acesso para um paciente já relacionado à clínica. A solicitação fica pendente; não libera histórico.

3. Paciente: entra com sua conta, lê o termo e decide. O servidor associa a decisão à identidade autenticada e preserva o termo apresentado.

4. Clínica B: consulta a linha do tempo. O backend verifica papel, vínculo, prazo e escopo, retorna somente registros permitidos e audita a leitura.

5. Revogação/vencimento: consultas seguintes deixam de receber registros externos sustentados pela autorização. Registros próprios continuam acessíveis ao profissional habilitado.

22.4 Contrato inicial de API

Todas as rotas abaixo possuem prefixo /api/v1/. Requisições e respostas utilizam JSON. Datas seguem ISO 8601 com fuso. Login utiliza username e password. Autenticação de usuário: Authorization: Bearer <access>.

• POST auth/login/: retorna access e refresh; POST auth/refresh/: renova access.

• GET me/: retorna usuário, patient_id quando aplicável e clínicas vinculadas.

• GET clinics/{clinic_id}/patients/: lista pacientes locais para médico habilitado.

• POST integration/records/: exige X-Integration-Key no formato UUID.segredo e JWT; corpo contém external_patient_id, external_id, occurred_at e summary. Retorna 201 ao criar, 200 no reenvio idêntico, 409 no conflito.

• POST clinics/{clinic_id}/grants/: corpo patient_id, purpose e expires_at; cria solicitação pendente e retorna termo, versão e destinatário.

• GET patient/grants/: lista somente autorizações do paciente autenticado.

• POST patient/grants/{grant_id}/decision/: decision=approve, deny ou revoke. Approve exige term_version correspondente ao termo apresentado.

• GET clinics/{clinic_id}/patients/{patient_id}/history/: retorna records, limit e external_access. Registros incluem origem, autoria, datas e resumo.

Erros: 400 para validação; 401 para autenticação ausente/inválida; 403 para vínculo ou acesso negado; 404 para recurso fora do contexto autorizado; 409 para conflito de estado ou reenvio divergente. O formato inicial é o padrão DRF: detail ou campos com listas de mensagens. Não há envelope personalizado de erros.

Listagens têm teto de 100 itens. Paginação, filtros por categoria e gestão de múltiplas integrações por fornecedor são evoluções. A chave de integração deve ficar no adaptador servidor em produção; sua entrada no navegador existe exclusivamente para demonstrar o fluxo com dados fictícios.

Critérios de validação

• Um atendimento de A chega ao HUB e preserva autoria e procedência.

• Reenvio idêntico não duplica; alteração do mesmo identificador retorna conflito.

• B não recebe histórico externo sem aprovação, com prazo vencido ou após revogação.

• Outro paciente não decide a autorização; recepção não consulta dados clínicos.

• Credencial de integração inválida e profissional de outra clínica não enviam registros.

• Registros recebidos após o aceite não aparecem sob o escopo inicial.

• Consulta permitida registra auditoria; login real emite JWT.

Parte 02 - Planejamento e acompanhamento

Etapa 1: alinhar escopo e contrato (todos), revisar arquitetura (Yasmin), construir e integrar base técnica (Carlos), atualizar documentação (Barbara e Toin). Etapa 2: validar cliente (Yasmin), backend e isolamento (Carlos e Toin), coerência documental (Toin e Barbara). Registrar datas exatas dos sábados e status no planejamento da equipe; o documento não presume prazos de calendário não confirmados.

Entregue nesta base: arquitetura revisada, contrato textual, backend inicial, migration, demonstração HTML/Fetch, configuração PostgreSQL e testes de regras. Em aberto: execução integrada no PostgreSQL, validação visual/manual da demonstração, adaptação das telas originais, cadastro/identificação real e integração com fornecedor externo. O relatório de testes e comandos de reprodução estão em docs/iniciar-backend.md.

O desenvolvimento deve ser validado durante a primeira etapa. A segunda etapa consolida evidências; não deve ser o primeiro momento em que frontend e backend se encontram.

Parte 00 - Histórico de versão

Versão 1.1, 11/10/2026, 6º período/Fase 02: reformulação da arquitetura para HUB de integração; clínica como tenant; identidade do paciente, autorizações com escopo/prazo e procedência; construção da base técnica e contrato inicial. O histórico não declara integração comercial nem conclusão de produção.

Decisões e próximas evoluções

• Copiar registros selecionados para o HUB nesta etapa, para consulta independente da disponibilidade momentânea da origem.

• Usar identificadores externos e vinculação provisionada no protótipo; identificação verificada e conciliação são próximas etapas.

• Manter registros recebidos sem edição pela API; modelar correções versionadas posteriormente.

• Adicionar adaptador de um fornecedor real somente após verificar acesso, contrato e formato dos dados.

• Produção requer validação institucional/jurídica da autorização eletrônica e das responsabilidades; o aceite de demonstração não comprova adequação jurídica.

Referências de implementação

Documentação Django 5.2: https://docs.djangoproject.com/en/5.2/

Autenticação DRF: https://www.django-rest-framework.org/api-guide/authentication/

Permissões DRF: https://www.django-rest-framework.org/api-guide/permissions/

As referências orientam implementação; não constituem certificação de segurança ou conformidade do projeto.